CHAPTER TWO

LITERATURE REVIEW

Dynamic Context-Aware Tries (DCA-Trie): Context-Conditioned Constrained Decoding for Knowledge Graph Question Answering

LEGEND / KEY FOR THIS DOCUMENT

📝 Writing Note — guidance on what content to include in each section
🖼️ Figure Suggestion — where a diagram or chart is required
∑ Equation / Model — where a formal mathematical expression must be presented
⚙️ Algorithm — where pseudocode or an algorithmic description is required

2.1 Introduction

📝 **Writing Note**	*Open by stating the purpose and scope of the chapter. State that the review is organised around ten thematic pillars that collectively establish the theoretical and empirical foundation for DCA-Trie: (1) hallucination in LLMs and its structural causes, (2) LLM reasoning mechanisms and their faithfulness limits, (3) knowledge graph question answering and multi-hop complexity, (4) constrained decoding frameworks, (5) beam search and prefix-based decoding mechanics, (6) retrieval-augmented generation and its structural limitations, (7) semantic relevance scoring with sentence transformers, (8) related work evaluated against DCA-Trie's design criteria, and (9) synthesis of research gaps and DCA-Trie positioning. End with a one-paragraph roadmap. Keep to one page maximum.*

2.2 Hallucination in Large Language Models: Taxonomy, Structural Causes, and the Scaling Paradox

📝 **Writing Note**	*This section establishes WHY the problem exists. It must be written as a systematic argument for why autoregressive LLMs structurally cannot avoid hallucination without external constraint mechanisms — not merely a description of what hallucination is. The addition of the scaling paradox subsection (2.2.3) is critical for journal-level credibility: it preempts the reviewer objection that simply using a larger model solves the problem.*

2.2.1 Defining Hallucination: A Formal Taxonomy for Knowledge-Intensive Generation

📝 **Writing Note**	*Define hallucination precisely using Ji et al. (2023) as the primary reference. Present Huang et al. (2023)'s taxonomy distinguishing: (1) intrinsic hallucination — the output directly contradicts the provided source material; (2) extrinsic hallucination — the output cannot be verified against any provided or external source. Critically extend this taxonomy to the KGQA setting: in KG-constrained generation, intrinsic hallucination means generating a triplet (e, r, e') that does not exist in the KG, and extrinsic hallucination means generating a reasoning chain that cannot be traced to any KG path. State that both types are unacceptable for safety-critical or knowledge-intensive applications. Cite Luo et al. (2024)'s finding that RAG-augmented approaches produce hallucinated reasoning steps in approximately one-third of cases even with supporting KG access — your strongest empirical justification for hard constraint mechanisms.*

2.2.2 The Autoregressive Generation Process as the Structural Root of Hallucination

📝 **Writing Note**	*Argue that hallucination is a structural property of autoregressive generation, not a correctable bug. Walk through the generation mechanism: at each step t, the model computes a probability distribution over the full vocabulary conditioned on the input and all previously generated tokens. No term in this expression corresponds to a factual verification mechanism. An error introduced at step t propagates forward as conditioning context for all subsequent tokens, producing grammatically fluent outputs whose factual incorrectness has no surface signal. Cite Brown et al. (2020) for the generation formalism, Huang et al. (2023) and Ji et al. (2023) for the error propagation argument.*

∑ **Equation / Model**	*Eq 2.1 — Autoregressive generation probability. Present: $P(y_t \mid y_{<t}, x)$ where x is the input prompt and $y_{<t} = (y_1, ..., y_{t-1})$ is the partially generated sequence. Annotate explicitly: (1) no term in this expression encodes a factual verification check; (2) the conditioning on $y_{<t}$ means errors at step t-1 propagate forward as context for all subsequent steps; (3) the distribution is over the full vocabulary $\|V\|$ — all tokens have nonzero probability at every step. This annotation is what makes the structural argument precise rather than merely qualitative.*

2.2.3 The Scaling Paradox: Why Larger Models Do Not Solve Hallucination

📝 **Writing Note**	*This subsection is essential for a top-tier journal. Preempt the reviewer objection that simply scaling model size eliminates hallucination. Cover: (1) TruthfulQA (Lin et al., 2022) — empirical evidence that larger models are not systematically more truthful, and in some cases exhibit inverse scaling on truthfulness benchmarks. (2) The sycophancy problem (Perez et al., 2022) — larger RLHF-tuned models are more likely to produce confident-sounding but incorrect statements that match user expectations. (3) The memorisation-generalisation tension — larger models memorise more training data, which can improve factual recall on seen facts but does not improve reasoning about compositional multi-hop questions unseen during training (Kandpal et al., 2023). (4) Cite Mallen et al. (2023) showing that even GPT-4 hallucinates on tail-entity questions. The conclusion: scaling addresses coverage over seen facts but does not address the structural absence of a verification mechanism in autoregressive generation. Hard constraints are necessary regardless of model scale.*

🖼️ **Figure**	*Fig 2.1 — Scaling vs truthfulness chart. A line chart with model parameter count on the x-axis (7B to 70B+) and TruthfulQA accuracy on the y-axis for several model families (GPT, Llama, Mistral). Show that accuracy does not monotonically improve with scale. Annotate the 'inverse scaling' region where larger models perform worse on truthfulness. Source: Lin et al. (2022) and subsequent evaluation papers. Tool: Python Matplotlib using published benchmark data.*

📝 **Writing Note**	*Conclude Section 2.2 with one crisp paragraph: 'These findings collectively establish that hallucination in LLMs is not a scale problem — it is a structural problem. Autoregressive generation provides no mechanism for grounding each generated token in a verifiable external source, and this absence cannot be remedied by increasing model capacity alone. The following sections review approaches that attempt to address this limitation, from soft grounding via retrieval augmentation through to the hard constraint mechanisms that motivate this dissertation.'*

2.3 Chain-of-Thought Reasoning, Self-Consistency, and the Faithfulness Ceiling of Unconstrained Multi-Step Generation

📝 **Writing Note**	*THIS IS A NEW SECTION — not in the previous version. It is critical for journal submission because your unconstrained CoT baseline is a primary comparison point in Chapter 4. A reviewer evaluating your results will immediately ask: why not just use a better prompting strategy? This section provides the answer: CoT dramatically improves multi-step reasoning accuracy but provides no structural faithfulness guarantee — which is precisely the property DCA-Trie targets. Without this section, the motivation for constrained decoding over CoT is not established in the literature review.*

2.3.1 Chain-of-Thought Prompting: Eliciting Step-by-Step Reasoning from LLMs

📝 **Writing Note**	*Introduce chain-of-thought prompting (Wei et al., 2022) as the dominant paradigm for improving multi-step reasoning in LLMs. Explain the mechanism: by including worked examples that show intermediate reasoning steps in the prompt, the LLM learns to produce its own intermediate steps before giving a final answer. Describe the emergent nature of CoT: Wei et al. (2022) show that CoT ability emerges at approximately 100B parameters and is negligible in smaller models. Present the empirical improvements: CoT prompting improves accuracy on GSM8K, MATH, and multi-hop QA benchmarks substantially over direct prompting. Then critically state the limitation: each reasoning step in a CoT chain is generated by the same autoregressive mechanism described in Section 2.2. An incorrect intermediate step produces a confident and fluent continuation that may result in a wrong final answer — and crucially, there is no mechanism to verify that any reasoning step corresponds to a verified KG fact.*

∑ **Equation / Model**	*Eq 2.2 — Chain-of-thought generation. Define CoT output as: (r_1, r_2, ..., r_k, a) where r_i are intermediate reasoning steps and a is the final answer. Each step is*

generated autoregressively: P(r_i | r_{<i}, x) following the same formulation as Eq 2.1. Annotate: no step r_i is constrained to correspond to a verified KG triplet. A hallucinated intermediate step r_i propagates forward as context for r_{i+1}, potentially amplifying the error. This is the CoT faithfulness ceiling.

2.3.2 Self-Consistency and Tree-of-Thought: Improving Reliability Without Improving Faithfulness

📝 **Writing Note**

Review the two major extensions of CoT that attempt to address its reliability limitations. (A) Self-consistency (Wang et al., 2023): sample multiple CoT chains and take the majority-vote answer. Improves accuracy by marginalising over reasoning paths but does not guarantee that any individual reasoning chain is factually grounded — a majority of hallucinated chains can still produce an incorrect majority-vote answer on multi-hop KGQA. (B) Tree-of-Thought (Yao et al., 2023): frames generation as a search over a tree of partial reasoning steps, using the LLM as both a generator and evaluator to prune unpromising branches. Improves on CoT for complex reasoning but still operates entirely within the LLM's parametric knowledge — no external verification of individual steps. For both methods, state explicitly: these approaches reduce the probability of hallucinated reasoning but cannot guarantee zero-hallucination outputs, because neither method modifies the token-level generation mechanism. This is the critical distinction from constrained decoding.

🖼️ **Figure**

Fig 2.2 — CoT vs constrained decoding faithfulness comparison. A 2x2 table: rows are CoT/Self-Consistency/ToT and KG-Constrained Decoding. Columns are: Multi-hop Accuracy (High/Low), Structural Faithfulness Guarantee (Y/N), External Verification Mechanism (Y/N), Parametric Knowledge Dependency (High/Low). Show that CoT-family methods score High on accuracy but N on faithfulness and Y on parametric dependency. Constrained decoding scores opposite. This table motivates the constrained decoding approach directly.

2.3.3 The Faithfulness Ceiling: Why Prompting Strategies Cannot Provide Structural Guarantees

📝 **Writing Note**

Make the formal argument for why no prompting strategy — however sophisticated — can provide the structural faithfulness guarantee that DCA-Trie targets. The argument: prompting strategies influence the probability distribution over generation outputs but do not modify the generation mechanism itself. Because the mechanism remains autoregressive token selection over the full vocabulary, any token in the vocabulary can in principle be selected at any step with nonzero probability. Structural faithfulness — defined as the guarantee that every generated token corresponds to a verified KG path — requires that some tokens have exactly zero probability at each step. This can only be achieved by modifying the generation mechanism directly, which is what constrained decoding does. Cite Geng et al. (2023) and Willard and Louf (2023) as the works that formalise this argument. Conclude: 'Prompting is a soft influence; constrained decoding is a hard mechanism. The former can improve faithfulness in expectation; only the latter can guarantee it with probability one.'

2.4 Knowledge Graph Question Answering: Structured Reasoning, Benchmark Design, and the Exponential Complexity of Multi-Hop Inference

📝 **Writing Note**	*This section establishes the technical domain in which DCA-Trie operates. Every design decision in Chapter 3 must be traceable to a fact established here. The section must be precise enough that a reviewer unfamiliar with KGQA can follow the technical arguments in Chapter 3 and 4 without additional background.*

2.4.1 Knowledge Graphs as Structured Repositories of Verified Facts: Formal Definition and Properties

📝 **Writing Note**	*Define a knowledge graph formally as a set of (entity, relation, entity) triplets forming a directed labelled graph. Give concrete examples from Freebase: (Marie Curie, award_received, Nobel_Prize_in_Physics), (Paris, capital_of, France). Describe Freebase (Bollacker et al., 2008) specifically: its scale (tens of millions of triplets), domain coverage (geography, history, entertainment, science), and its status as the underlying resource for WebQSP and CWQ. Critically distinguish a KG from a text corpus: every triplet in a KG is a human-verified fact, machine-readable, and unambiguously structured. These properties are what make KG-constrained generation possible — the constraint oracle can verify any candidate token against the KG with a deterministic lookup rather than a probabilistic semantic matching operation.*

∑ **Equation / Model**	*Eq 2.3 — Formal KG and path definition. Define: $G = (E, R, T)$ where E is the entity set, R is the relation set, and $T \subseteq E \times R \times E$ is the set of verified triplets. A KG path of length L is a sequence $p = (e_0, r_1, e_1, r_2, e_2, ..., r_L, e_L)$ where each consecutive triplet $(e_{i-1}, r_i, e_i) \in T$. The answer to a multi-hop question q is the terminal entity e_L of the correct reasoning path. This formal definition is load-bearing: DCA-Trie's faithfulness guarantee is stated in terms of this definition — every generated token must correspond to a valid prefix of some path p in G.*

🖼️ **Figure**	*Fig 2.3 — Knowledge graph example subgraph. Draw a directed graph with 5–6 entity nodes and labelled relation edges. Include the example from Chapter 1: Elvis_Presley --[starred_in]--> Blue_Hawaii --[directed_by]--> Norman_Taurog --[nationality]--> American. Label each edge with its Freebase relation string. Annotate that each edge = one verified triplet in T. Tool: draw.io.*

2.4.2 WebQSP and ComplexWebQuestions: Benchmark Architecture, Evaluation Protocol, and Hop-Depth Stratification

📝 **Writing Note**	*Describe both benchmarks with precision. WebQSP (Yih et al., 2016): derived from WebQuestions, ~4,700 questions, grounded in Freebase, 1–2 hop depth, annotated with SPARQL queries, evaluated using Hits@1 and F1. CWQ (Talmor and Berant, 2018): constructed by composing and transforming WebQSP questions into more complex forms, ~34,000 questions, up to 4 hops, substantially harder. Explain WHY*

these two benchmarks together provide comprehensive evaluation: WebQSP represents the regime where constrained decoding should perform near-perfectly (shallow reasoning, small valid path set); CWQ represents the regime where the permissiveness problem is most acute (deep reasoning, exponentially large valid path set). Explain the hop-depth stratified evaluation protocol from Lan et al. (2022) — this is the protocol adopted in Chapter 4, so it must be established here.

🖼️ Figure

Fig 2.4 — Benchmark statistics and hop-depth distribution. Two side-by-side bar charts. LEFT: WebQSP question count by hop depth (1-hop, 2-hop). RIGHT: CWQ question count by hop depth (1, 2, 3, 4). Annotate the percentage of questions at each depth. Show that the majority of CWQ questions involve 3–4 hops — the hardest setting. Source: Talmor and Berant (2018) and Lan et al. (2022). Tool: Python Matplotlib.

2.4.3 Multi-Hop Reasoning Complexity: Exponential Path Growth and the Permissiveness Problem Quantified

📝 Writing Note

Formally establish why multi-hop questions are qualitatively harder than single-hop questions, and derive the quantitative basis for the permissiveness problem. A 1-hop question requires selecting one correct relation from the neighbours of the question entity — the valid set is small. A 3-hop question requires selecting a correct sequence across three steps, where at each step the number of structurally valid continuations multiplies. Derive the exponential growth formula and instantiate it for Freebase. Then connect directly to the permissiveness problem: for a 3-hop CWQ question, the static KG-Trie in GCR may contain thousands of valid paths, of which only one is correct. At every generation step, the LLM must discriminate among thousands of valid-but-irrelevant paths using its parametric knowledge — precisely the mechanism that constrained decoding was designed to suppress. This is the quantitative motivation for DCA-Trie's semantic filtering. Cite He et al. (2021) and Lan et al. (2022) for empirical performance degradation with hop depth.

∑ Equation / Model

Eq 2.4 — Exponential path growth with hop depth. Define: $|P_L| \leq |E_q| \times d^L$ where $|P_L|$ is the upper bound on valid paths of length L, $|E_q|$ is the number of question entities, and d is the average out-degree of entities in G. For Freebase: $d \approx 20$, $|E_q| = 1–3$. At L=3: $|P_3| \leq 3 \times 8{,}000 = 24{,}000$ valid paths. At L=4: $|P_4| \leq 3 \times 160{,}000$. Only one path is correct. The Semantic Irrelevance Ratio (SIR) introduced in Chapter 3 quantifies the proportion of these paths that are semantically irrelevant to the question.

🖼️ Figure

Fig 2.5 — Multi-hop path explosion tree. A tree rooted at the question entity. Level 1: 8–10 relation/entity branches. Level 2: each expands further. Level 3: same. Highlight the single correct path in teal. Mark all other paths in gray. Annotate the total number of valid paths at depth 3 using the formula from Eq 2.4. This is the key visual argument for DCA-Trie — the correct path is buried in thousands of valid-but-wrong alternatives. Tool: draw.io.

2.5 Constrained Decoding for Faithful Text Generation: From Logit Masking to

Knowledge Graph Oracles

📝 **Writing Note**	*This is the most technically dense and most important section of the literature review. Every framework reviewed here is either a direct baseline or a prior work whose specific limitation DCA-Trie addresses. Be precise and critical throughout. The section builds progressively from the foundational logit-masking mechanism through grammar constraints to KG-specific frameworks, creating a clear technical lineage that positions DCA-Trie as the natural next step.*

2.5.1 The Logit-Masking Paradigm: Hard Constraints on Autoregressive Vocabulary Selection

📝 **Writing Note**	*Introduce the general logit-masking paradigm as the foundational mechanism for hard-constrained generation. Explain the key insight: rather than influencing the probability distribution over tokens (as prompting does), a constraint oracle computes a binary mask that sets the logit of every invalid token to negative infinity (effectively zero probability after softmax) before token selection. This makes generation of invalid tokens mathematically impossible rather than merely improbable. Introduce the masking formulation from Geng et al. (2023) and Willard and Louf (2023). Then introduce the central design question that distinguishes all subsequent frameworks: what function computes the mask m_t? GCR answers $f(G, E_q)$. DCA-Trie answers $f(G, E_q, q, y_{<t})$. Every subsequent subsection is a step in the progression toward this answer.*

∑ **Equation / Model**	*Eq 2.5 — Logit masking formulation. Present: $\tilde{a}_t = m_t \odot \alpha_t$ where $\alpha_t \in \mathbb{R}^{\lvert V\rvert}$ is the raw logit vector at decoding step t, $m_t \in \{0,1\}^{\lvert V\rvert}$ is the binary validity mask computed by the constraint oracle, and $\odot$ is element-wise product. Token selection under greedy decoding: $y_t = \mathrm{argmax}(\tilde{a}_t)$. Under beam search with beam width k: maintain top-k sequences ranked by cumulative log-probability of masked logits. Annotate: the central research question of this dissertation is how m_t should be computed — the answer determines the expressiveness, faithfulness, and efficiency of the framework.*

🖼️ **Figure**	*Fig 2.6 — Logit masking mechanism. A horizontal bar chart showing raw logit vector α_t over a subset of vocabulary tokens. Below it, the binary mask m_t (1 = valid, 0 = invalid). Below that, the masked logit vector $\tilde{a}_t$ where invalid tokens are zeroed. Annotate that token selection operates only over the non-zero (valid) region. Show that DCA-Trie's contribution is to make the valid region semantically focused rather than structurally exhaustive. Tool: Python Matplotlib or PowerPoint.*

2.5.2 Grammar-Constrained Decoding: GCD and Outlines as Foundational Format Constraints

📝 **Writing Note**	*Review Geng et al. (2023) GCD and Willard and Louf (2023) Outlines as the foundational works on structure-constrained LLM generation. GCD constrains generation to strings conforming to a context-free grammar using an Earley parser as the constraint oracle. Outlines uses finite-state automata for regular expression and JSON schema constraints. Both precompute constraint structures — parse*

states or automaton transitions — that can be evaluated efficiently at inference time. The Outlines paper also introduces the key efficiency insight that DCA-Trie must contend with: precomputing constraint structures amortises the cost across generation steps, making constrained decoding practical. Critically state the shared limitation: both frameworks constrain the FORMAT of output (grammar, schema, structure) but not the FACTUAL CONTENT. A grammatically valid JSON response can contain entirely hallucinated facts. This motivates the transition from format constraints to knowledge graph constraints.

⚙️ Algorithm

Alg 2.1 — Grammar-constrained decoding (conceptual, after Geng et al., 2023). INPUT: context-free grammar G_CFG, LLM with vocabulary V, prompt x. PRECOMPUTE: Earley parse automaton A from G_CFG. GENERATION: FOR each step t from 1 to max_length: (1) query A with partial string y_{<t} to obtain valid next tokens: V_t = {v ∈ V : y_{<t} + v is a valid grammar prefix}. (2) compute mask: m_t[v] = 1 if v ∈ V_t, else 0. (3) apply mask: ã_t = m_t ⊙ LLM_logits(y_{<t}, x). (4) select: y_t = argmax(ã_t). RETURN: y_1...y_T. Annotate: V_t depends only on the partial string and the grammar — no dependence on question semantics. This is the same structural limitation present in GCR's KG-Trie.

2.5.3 Graph-Constrained Reasoning (GCR): Structural Faithfulness via KG-Trie Oracles

📝 Writing Note

Present GCR (Luo et al., 2024) in full technical detail — this is your primary baseline. Cover precisely: (1) Entity linking — question entities E_q are identified using an entity linker before generation begins. (2) KG-Trie construction — BFS from E_q up to L hops, encoding all reachable KG paths as a prefix tree. The trie node at depth d corresponds to a partial path of d tokens. (3) Constrained decoding — at each generation step, the trie is queried with the current partial generation y_{<t} to return V_t, the set of tokens that extend a valid trie prefix. The mask m_t is constructed from V_t. (4) Faithfulness guarantee — because every token must correspond to a valid trie prefix, and every trie path corresponds to a verified KG path in T, structural hallucination is mathematically impossible. (5) Reported performance — 92.6 Hits@1 on WebQSP and 75.8 Hits@1 on CWQ with 100% structural faithfulness on the Llama-3.1-8B backbone. Then state the permissiveness problem precisely: the KG-Trie is a function f(G, E_q) evaluated once, before generation. The valid set V_t is therefore identical at step 1 and step 10 of generation, regardless of what has been generated. For a 3-hop CWQ question, this trie may contain thousands of valid paths, forcing the LLM to use its hallucination-prone parametric knowledge to discriminate among them.

∑ Equation / Model

Eq 2.6 — GCR valid token set. Define: W_val^GCR = f(G, E_q). The trie T is constructed as: T = BFS(G, E_q, L) where L is the maximum hop depth. At each step t: V_t = {v ∈ V : prefix(y_{<t}) + v ∈ T}. Critical annotation: W_val^GCR has NO dependence on q (question text) or y_{<t} (partial generation state). The valid set is semantically blind and computationally fixed before generation begins. This is the exact formulation that DCA-Trie upgrades to W_val^DCA(t) = f(G, E_q, q, y_{<t}).

🖼️ Figure

Fig 2.7 — GCR KG-Trie structure. A prefix tree rooted at the question entity (e.g. Elvis_Presley). First level: all relation tokens outgoing from E_q (starred_in, born_in, nationality, ...). Second level: entity tokens reachable via each relation. Third level:

	further relations. Highlight in red that ALL structurally valid paths are present regardless of question semantics — the permissiveness problem. Annotate the trie size at depth 3 using the formula from Eq 2.4. Tool: draw.io.

2.5.4 Decoding on Graphs (DoG): Step-Wise Topological Expansion and Its Persistent Limitations

📝 **Writing Note**	*Present Li et al. (2024) DoG as a partial remedy to GCR's static trie. DoG introduces step-wise trie expansion: after committing to an entity e_t, the valid set is extended with all KG neighbours of e_t. This makes the constraint topologically dynamic — it evolves as reasoning progresses through the graph. Evaluate DoG's two genuine improvements over GCR: (1) the valid set at step t reflects the entities already visited, so the trie cannot contain paths that are structurally inconsistent with the committed reasoning direction; (2) the dynamic expansion can handle graphs that are too large to enumerate statically. Then state DoG's two persistent limitations with precision: (1) the expansion rule is semantically blind — every neighbour of every visited entity is admitted regardless of semantic relevance to q or y_{<t}, so the permissiveness problem persists; (2) DoG sacrifices the computational efficiency of precomputed trie structures because the trie must be rebuilt at every step, contrary to the efficiency insight of Willard and Louf (2023). State explicitly that DCA-Trie addresses both limitations: the expansion rule is conditioned on semantic relevance, and static filtering (DCA-Trie v1) preserves the efficiency of precomputed structures.*

∑ **Equation / Model**	*Eq 2.7 — DoG step-wise expansion rule. At step t, after committing entity e_t: T_{t+1} = T_t ∪ {(e_t, r, e') : (e_t, r, e') ∈ T for all r, e'}. Annotate: the expansion set {(e_t, r, e')} contains ALL KG neighbours of e_t with no semantic filtering. Compare with DCA-Trie v2 expansion (defined in Chapter 3): T_{t+1} = T_t ∪ {(e_t, r, e') : score(e', q, y_{<t}) ≥ τ}. The single change — adding the semantic score condition — is the core algorithmic contribution of DCA-Trie v2.*

🖼️ **Figure**	*Fig 2.8 — Three-way constraint evolution comparison. Three columns labelled GCR, DoG, DCA-Trie. Three rows labelled Step t=1, Step t=2, Step t=3. Each cell shows a circle whose diameter represents the valid token set size, with a small inner teal circle representing the semantically relevant subset. GCR: all circles same large size (static). DoG: circles grow at each step but remain large (topologically dynamic but semantically blind). DCA-Trie: circles are smaller and focused at every step (semantically filtered). Annotate the teal inner circle as the 'relevant' subset. This figure is publishable as a standalone visual argument for DCA-Trie. Tool: draw.io or PowerPoint.*

2.6 Beam Search Decoding and Prefix-Tree Constraint Integration: Mechanics, Efficiency, and the Step-Wise Constraint Problem

📝 **Writing Note**	*THIS IS A NEW SECTION — not in the previous version. It is necessary because DCA-Trie v2 operates step-wise during beam search, and reviewers evaluating your method chapter need to understand both the beam search mechanism and how trie-*

based constraints interact with beam expansion before they can follow the DCA-Trie design. Without this section, the step-wise constraint update in DCA-Trie v2 (and its efficiency implications) cannot be properly motivated or understood.

2.6.1 Beam Search: Approximate Global Optimisation Over Autoregressive Sequences

📝 Writing Note

Introduce beam search as the standard approximate decoding algorithm for autoregressive sequence generation. Explain the mechanism: maintain a set of k partial sequences (the beam) at each generation step. At step t, expand each beam hypothesis by all vocabulary tokens, compute the masked logit scores, and retain only the top-k hypotheses by cumulative log-probability. The beam width k controls the exploration-exploitation tradeoff: k=1 is greedy decoding; larger k explores more of the generation space at the cost of k-fold compute. Present the beam search algorithm formally. Explain why beam search is used in GCR and why it matters for DCA-Trie: with a binary validity mask, beam search over the valid token set is equivalent to best-first search over the KG-Trie — the beam hypotheses at each step correspond to the k most probable partial KG paths. This connection is what makes beam-search-based constrained decoding both correct and efficient.

∑ Equation / Model

Eq 2.8 — Beam search with masked logits. At step t, for each beam hypothesis $h \in B_t$: $score(h + v) = score(h) + \log(softmax(\tilde{a}_t)[v])$ for all $v \in V$ with $\tilde{a}_t[v] > 0$. New beam: B_{t+1} = top-k over all (h, v) pairs by score(h + v). State: when combined with the logit mask from Eq 2.5, beam search explores only the valid subspace of the vocabulary at each step. The KG-Trie defines this valid subspace. DCA-Trie's semantic filter narrows the subspace further, reducing the number of (h, v) pairs that must be scored at each beam expansion step.

2.6.2 Prefix Trees as Constraint Oracles: Efficiency, Precomputation, and the Dynamic Rebuilding Cost

📝 Writing Note

Explain how prefix trees (tries) function as constraint oracles for beam-search decoding. A trie over KG paths supports O(d) lookup at each step — given the current partial sequence $y_{<t}$, the set of valid next tokens is exactly the set of children of the current trie node. This is the efficiency insight of Willard and Louf (2023): by precomputing the trie, the constraint oracle can be queried in constant time at each generation step. Present the trie construction complexity: $O(|P| \times L)$ where $|P|$ is the number of valid paths and L is the maximum path length. Then analyse the tradeoff introduced by DoG's step-wise expansion: rebuilding the trie at every step costs $O(|N_t| \times d)$ at each step t where N_t is the set of newly reachable entities — not prohibitive per step, but it accumulates over L steps and eliminates the precomputation benefit. DCA-Trie v1 (static semantic filtering at construction time) preserves the precomputation benefit; DCA-Trie v2 (dynamic step-wise expansion with semantic filtering) rebuilds at each step but with a much smaller expansion set than DoG because the semantic filter prunes most candidates.

🖼️ Figure

Fig 2.9 — Trie operations and beam search interaction. A diagram showing: (1) at the top, a partial beam hypothesis $y_{<t}$; (2) trie lookup returning valid next tokens V_t; (3) beam expansion: each beam hypothesis × $|V_t|$ candidates, scored and top-

k retained. Show how DCA-Trie's semantic filter reduces |V_t| (fewer candidates to score per beam step), resulting in faster beam expansion. Annotate the computational savings. Tool: draw.io.

2.6.3 The Step-Wise Constraint Problem: When the Constraint Must Know What Has Been Generated

📝 **Writing Note**

Introduce the step-wise constraint problem as the core design challenge that motivates DCA-Trie. In GCR, the constraint is computed once before generation and queried at each step — the oracle has no memory of what has been generated beyond the structural consistency enforced by the trie. In DCA-Trie v2, the oracle is updated after each committed entity to reflect the semantic direction of the reasoning chain. This requires the oracle to maintain state across generation steps — specifically, the partial generation y_{<t} and the question embedding must be re-incorporated at each update. Present this as a formal requirement: the constraint oracle must be a function of (G, E_q, q, y_{<t}) not just (G, E_q). This is the formal upgrade from GCR to DCA-Trie stated as a design requirement, before the implementation is presented in Chapter 3.

∑ **Equation / Model**

Eq 2.9 — The DCA-Trie constraint oracle requirement. Formally state: W_val^DCA(t) = f(G, E_q, q, y_{<t}) where the function f must satisfy: (i) structural faithfulness: every token v ∈ W_val^DCA(t) corresponds to a valid KG path prefix; (ii) semantic relevance: the proportion of W_val^DCA(t) consisting of semantically irrelevant paths is strictly less than that of W_val^GCR; (iii) recall preservation: the gold answer path is never excluded from W_val^DCA(t) (false negative rate < 5% on validation set). These three conditions are the formal specification of DCA-Trie, evaluated quantitatively in Chapter 4.

2.7 Retrieval-Augmented Generation: Architectural Variants, KG-Augmented Approaches, and the Hard Limit of Soft Grounding

📝 **Writing Note**

This section is substantially expanded from the previous version. A top-tier reviewer will expect engagement with the latest RAG variants — GraphRAG, Self-RAG, FLARE, IRCoT — not just the 2020 Lewis et al. paper. The section must still conclude with the same argument: no RAG variant provides structural faithfulness guarantees. But the argument is much stronger when you have systematically addressed the strongest modern alternatives.

2.7.1 The RAG Paradigm: Dense Retrieval, Fusion-in-Decoder, and the Soft Grounding Ceiling

📝 **Writing Note**

Introduce RAG (Lewis et al., 2020) as the dominant paradigm for grounding LLM outputs in external knowledge. Explain the mechanism: a dense retriever (typically DPR, Karpukhin et al., 2020) selects relevant passages based on embedding similarity, and an LLM reader generates the answer conditioned on the retrieved passages and the question. Fusion-in-Decoder (Izacard and Grave, 2021) extends

this by encoding all retrieved passages independently and fusing their representations in the decoder. Critically state the shared limitation: retrieved context is a SOFT influence — the LLM is free to generate any token regardless of the retrieved material. The retriever controls WHAT the LLM sees; the constraint oracle controls WHAT the LLM may generate. RAG addresses the former; constrained decoding addresses the latter.

2.7.2 KG-Augmented Retrieval: Structured Context Without Structural Guarantees

📝 **Writing Note**

Review KG-augmented RAG approaches that retrieve KG subgraphs rather than text passages, as these are the closest RAG variants to constrained KG decoding. Cover: He et al. (2021) DAGQA — uses attention over retrieved KG subgraphs to guide multi-hop reasoning; Yasunaga et al. (2021) QA-GNN — combines text and KG context via graph neural networks; Shi et al. (2021) — retrieves KG paths as context strings. For each: state the faithfulness limitation precisely. In DAGQA and QA-GNN, the retrieved KG subgraph is provided as context — the LLM can attend to it, ignore it, or generate facts that go beyond it. The KG is advisory, not constraining. This is the critical architectural distinction from GCR and DCA-Trie.

2.7.3 Advanced RAG Variants: Self-RAG, FLARE, IRCoT, and GraphRAG

📝 **Writing Note**

This subsection is new and critical for journal credibility. Review four advanced RAG variants that a reviewer may cite as alternatives to constrained decoding: (A) Self-RAG (Asai et al., 2024) — adds reflection tokens to control when retrieval is triggered and whether retrieved passages are relevant or supportive. Improves retrieval precision but does not constrain generation — the LLM remains free to generate any token. (B) FLARE (Jiang et al., 2023) — triggers retrieval when the LLM's next-token probability is low, using generation confidence as a retrieval signal. Addresses the WHEN to retrieve problem but not the generation faithfulness problem. (C) IRCoT (Trivedi et al., 2022) — interleaves retrieval with chain-of-thought reasoning steps, retrieving at each reasoning step. Improves multi-hop accuracy but each CoT step is still unconstrained. (D) GraphRAG (Edge et al., 2024) — uses KG-structured communities for retrieval, improving coverage of complex relational queries. Strongest RAG-based competitor: explicitly uses graph structure. Critically: GraphRAG retrieves graph context but does not constrain decoding. A generation conditioned on GraphRAG context can still produce tokens that do not correspond to any KG triplet. For every variant, state one sentence: 'Unlike DCA-Trie, [variant] does not modify the token selection mechanism and therefore cannot guarantee structural faithfulness.'

🖼️ **Figure**

Fig 2.10 — Advanced RAG vs constrained decoding comparison. Two-panel figure. LEFT: RAG pipeline (any variant) — retrieved context flows into LLM as additional input; output arrow labelled 'any token in V possible'. RIGHT: constrained decoding pipeline — constraint oracle produces binary mask applied to logits; output arrow labelled 'only valid KG-path tokens possible'. Annotate in red on the LEFT: 'soft influence — faithfulness not guaranteed.' Annotate in green on the RIGHT: 'hard constraint — structural faithfulness guaranteed.' Tool: draw.io or PowerPoint.

2.7.4 The Formal Impossibility of Structural Faithfulness Under Soft Grounding

📝 Writing Note	*Make the formal impossibility argument concisely and precisely. Claim: no retrieval-augmented approach can provide structural faithfulness guarantees as defined in this dissertation. Proof sketch: structural faithfulness requires that every generated token corresponds to a verified KG triplet. This requires that every non-KG-valid token has exactly zero probability at each generation step. Under any RAG approach, the LLM's raw logit distribution assigns nonzero probability to all tokens (or at least all tokens not explicitly masked to -inf). Retrieval modifies the input to the LLM but does not modify the logit computation mechanism. Therefore, for any RAG approach, there exists a nonzero probability of generating a non-KG-valid token at any step — structural faithfulness with probability one is not achievable. Only a constraint oracle that explicitly sets logits of invalid tokens to -inf can achieve probability-one faithfulness. This is what GCR, DoG, and DCA-Trie provide. Cite Luo et al. (2024) for the empirical comparison: GCR achieves 100% structural faithfulness where RAG-based approaches do not.*

2.8 Semantic Relevance Scoring with Sentence Transformer Embeddings: Training Objectives, Architecture Selection, and Application to Constraint Oracle Construction

📝 Writing Note	*This section is substantially expanded from the previous version. It now covers the SBERT training objective equation, a systematic comparison of encoder alternatives, and a clear argument for why cosine similarity over sentence transformer embeddings is the right choice for DCA-Trie's relevance scorer. Every design choice in Section 2.8 must be justifiable to a reviewer who asks: 'Why not use BM25? Why not use the LLM's own hidden states?'*

2.8.1 Sentence-BERT: Training Objective, Architecture, and the Semantic Similarity Property

📝 Writing Note	*Introduce sentence transformers (Reimers and Gurevych, 2019) — specifically the SBERT architecture — as models trained to produce dense vector representations where semantically similar sentences have high cosine similarity. Explain the architecture: a BERT-like transformer encoder with mean pooling over token representations to produce a fixed-length sentence embedding. Describe the training objective: a siamese network trained on sentence pairs using either a regression objective (minimise MSE against human similarity scores) or a classification objective (predict entailment/contradiction/neutral using softmax over concatenated and differenced representations). The critical property for DCA-Trie: after SBERT training, semantically similar sentences cluster together in the embedding space and semantically dissimilar sentences are pushed apart — enabling cosine similarity to serve as a reliable proxy for semantic relevance without task-specific fine-tuning.*

∑ Equation / Model	*Eq 2.10 — SBERT training objective (classification formulation). Define the sentence pair loss: $L = \text{CrossEntropy}(\text{softmax}(W_t \times (u, v, \lvert u-v \rvert)), y)$ where $u = E(s_1)$ and $v = E(s_2)$ are the sentence embeddings of a pair, $(u, v, \lvert u-v \rvert)$ is their concatenation with element-wise difference, W_t is a trainable classification head, and $y \in$ {entailment,*

neutral, contradiction}. This objective pulls embeddings of entailed pairs together and pushes contradiction pairs apart. At inference time, cosine_similarity(E(s_1), E(s_2)) measures the degree of semantic alignment without requiring the classification head.

2.8.2 Encoder Architecture Selection: all-MiniLM-L6-v2 vs Alternatives

📝 **Writing Note**

Justify the selection of all-MiniLM-L6-v2 over alternatives through systematic comparison. Cover: (A) all-MiniLM-L6-v2 — 6-layer MiniLM distilled from a larger teacher, 384-dimensional embeddings, ~22M parameters, ~80ms inference on CPU. Strong semantic performance on STS benchmarks with the lowest inference latency among competitive models. (B) all-mpnet-base-v2 — stronger semantic performance but ~110M parameters and 4x slower inference. For iterative beam-search decoding where the scorer is called at every step, this latency accumulates unacceptably. (C) BGE-M3 (Chen et al., 2024) — state-of-the-art multilingual retrieval encoder. Stronger performance than MiniLM but 568M parameters — not feasible for step-wise decoding on a single A100. (D) LLM hidden states as relevance scorer — using the LLM's own last-layer representations. Circular: the LLM's hidden states encode the same parametric biases that cause hallucination. Using them as a relevance filter does not introduce external grounding. (E) BM25 — lexical retrieval, fast and interpretable but unable to capture semantic similarity between paraphrase-equivalent paths. Conclude: all-MiniLM-L6-v2 provides the optimal balance of semantic quality and inference speed for iterative step-wise constraint filtering.

🖼️ **Figure**

Fig 2.11 — Encoder alternatives comparison table. Columns: Encoder, Parameters, Embedding Dim, Inference Latency (ms/query), STS-B Score, Suitable for Step-Wise Decoding (Y/N). Rows: all-MiniLM-L6-v2 (selected), all-mpnet-base-v2, BGE-M3, LLM hidden states, BM25. Highlight all-MiniLM-L6-v2 row in pale blue. The last column shows why it is the only suitable option for iterative decoding at beam-search speed.

2.8.3 Cosine Similarity as a Hard Constraint Signal: From Soft Retrieval to Binary Admission

📝 **Writing Note**

The key conceptual contribution of this subsection: explain the distinction between using cosine similarity as a soft retrieval ranking signal (standard in the literature) versus using it as a hard binary admission criterion (DCA-Trie's approach). In standard dense retrieval, cosine similarity produces a continuous relevance score used to rank candidates — high-scoring candidates are retrieved, low-scoring ones are not, but there is no hard cutoff. DCA-Trie applies cosine similarity as a threshold function: paths with score(p, q, y_{<t}) ≥ τ are admitted to the valid set (mask value 1); paths below τ are excluded (mask value 0). This binary admission converts the soft retrieval signal into a hard constraint on the generation mechanism. The threshold τ is the key hyperparameter controlling the precision-recall tradeoff of the relevance filter — its selection is described in Chapter 3 and evaluated in Chapter 4. Cite the distinction between soft and hard grounding from Section 2.7 to reinforce the conceptual consistency of the design.

∑ **Equation /**

Eq 2.11 — DCA-Trie semantic relevance score and admission criterion. Define:

Model	*score(p, q, y_{<t}) = cos(E(p), E(concat(q, y_{<t}))) where E(·) is the all-MiniLM-L6-v2 encoder, p is a candidate KG path expressed as a natural language string 'e_0 r_1 e_1 r_2 e_2...', and concat(q, y_{<t}) is the question concatenated with the partial generation. Admission criterion: admit p to W_val^DCA(t) if and only if score(p, q, y_{<t}) ≥ τ. Annotate: this converts a continuous relevance score into a binary mask entry, integrating semantic relevance directly into the logit-masking framework from Eq 2.5.*

🖼️ **Figure**	*Fig 2.12 — Semantic scoring and admission pipeline. Full end-to-end diagram: (1) Input question q + partial generation y_{<t} → SBERT encoder → query vector. (2) Candidate KG path p (as text string) → same SBERT encoder → path vector. (3) Cosine similarity → continuous score in [0,1]. (4) Threshold τ → binary admission decision (admit/reject). (5) Admitted paths form the valid set W_val^DCA(t). (6) Valid set → binary mask m_t → applied to LLM logits. This diagram connects the semantic scorer directly to the logit-masking framework from Section 2.5. Tool: draw.io or PowerPoint.*

2.9 Related Work

📝 **Writing Note**	*Keep this section compact — approximately one and a half pages in the final paper. Sections 2.2–2.8 have already surveyed the landscape thematically. This section's sole purpose is to position DCA-Trie against the specific papers a reviewer might cite as prior art. Three subsections only. For every paper: one sentence on what it does, one sentence on its result, one sentence on the precise gap it leaves that DCA-Trie addresses. No paper should receive more than three sentences. Do not repeat arguments already made in earlier sections — reference them by number instead.*

2.9.1 KG-Constrained Decoding: GCR, DoG, and the Unresolved Permissiveness Problem

📝 **Writing Note**	*This is your primary related work subsection — the two works most directly related to DCA-Trie. GCR (Luo et al., 2024): constructs a static KG-Trie from f(G, E_q) before generation, achieving 92.6 Hits@1 on WebQSP and 100% structural faithfulness, but the valid token set is semantically blind and identical at every generation step. DoG (Li et al., 2024): extends GCR with step-wise topological trie expansion after each committed entity, improving dynamic responsiveness at the cost of per-step reconstruction, but the expansion rule admits all KG neighbours regardless of semantic relevance. Both works leave the permissiveness problem unresolved: the valid token set contains thousands of structurally correct but contextually irrelevant paths at every generation step, forcing the LLM to rely on its hallucination-prone parametric knowledge for semantic discrimination. DCA-Trie directly addresses this by conditioning the valid token set on both the input question semantics and the partial generation, formally upgrading the constraint oracle from f(G, E_q) to f(G, E_q, q, y_{<t}). End with a one-sentence transition: 'DCA-Trie is designed as a direct extension of GCR and DoG, preserving their structural faithfulness guarantee while resolving the permissiveness problem neither addresses.'*

Figure	*Fig 2.13 — Primary related work comparison table. Rows: GCR (Luo et al. 2024), DoG (Li et al. 2024), DCA-Trie v1 (proposed), DCA-Trie v2 (proposed). Columns: Oracle Construction, Structural Faithfulness, Semantic Conditioning (Y/N), Dynamic Update (Y/N), Precomputed Efficiency (Y/N), Permissiveness Quantified (Y/N), Hits@1 WebQSP. This is the primary publishable comparison table — it is Table 1 of the journal paper. GCR: f(G,Eq) ✓ × × ✓ × 92.6. DoG: f(G,Eq,t) ✓ × ✓ × × —. DCA-Trie v1: f(G,Eq,q) ✓ ✓ × ✓ ✓ TBD. DCA-Trie v2: f(G,Eq,q,y<t) ✓ ✓ ✓ ◗ ✓ TBD.*

2.9.2 Hallucination Mitigation and KGQA: The Soft-to-Hard Spectrum

Writing Note	*Cover all remaining related work in two compact clusters. (A) Soft grounding approaches — RAG (Lewis et al., 2020), Self-RAG (Asai et al., 2024), GraphRAG (Edge et al., 2024), DAGQA (He et al., 2021), QA-GNN (Yasunaga et al., 2021), self-consistency (Wang et al., 2023). One paragraph maximum. For all: state that retrieved context and post-hoc verification improve faithfulness in expectation but cannot provide probability-one structural faithfulness guarantees because none modifies the token selection mechanism (as established in Sections 2.3 and 2.7). (B) Format-constrained decoding — GCD (Geng et al., 2023), Outlines (Willard and Louf, 2023), entity-constrained generation (Cao et al., 2021; De Cao et al., 2021). One paragraph maximum. State that these works establish the logit-masking paradigm on which DCA-Trie builds, but they constrain output format or entity vocabulary rather than multi-hop KG path faithfulness — the constraint oracle is a function of a grammar or entity list, not of a knowledge graph with relational structure. DCA-Trie inherits the logit-masking mechanism from this line of work and extends it to semantically conditioned KG path constraints.*

2.9.3 Semantic Similarity for Graph Reasoning: The Retrieval-Constraint Boundary

Writing Note	*Cover works that use semantic similarity signals in graph-based reasoning — the closest conceptual prior art to DCA-Trie's scoring mechanism. Cover: path retrieval for KGQA (Shi et al., 2021; Saxena et al., 2022), attention-guided graph traversal (He et al., 2021 DAGQA), semantic path reranking approaches. For each, state the precise boundary in one sentence: '[Work] uses semantic similarity as a [retrieval / reranking / attention] signal that influences which paths the LLM sees or which answer is selected after generation — it does not apply semantic similarity as a hard binary admission criterion that determines which tokens the LLM is permitted to generate.' This distinction — soft signal vs hard constraint — is the exact novelty boundary of DCA-Trie. Conclude the section with one synthesis paragraph: 'No prior work combines semantic relevance scoring with hard constraint oracle construction. Works that provide structural faithfulness (GCR, DoG) do not condition on semantics. Works that use semantic similarity (path retrieval, reranking) do not provide structural faithfulness. DCA-Trie is the first framework to achieve both simultaneously, and the Semantic Irrelevance Ratio introduced in Chapter 3 is the first metric designed to measure this property directly.'*

2.10 Synthesis of the Literature: Three Unresolved Gaps and the DCA-Trie Contribution

📝 **Writing Note**	*This is the most important section in Chapter 2. It distils the entire chapter into three precise gap statements and maps each one to a DCA-Trie component and project objective. Keep it tight — approximately one page of prose plus two figures. Every sentence must be citable, falsifiable, and directly connected to an experiment in Chapter 4.*

2.10.1 Three Unresolved Gaps at the Intersection of Structural Faithfulness, Semantic Relevance, and Constraint Measurability

📝 **Writing Note**	*State three gaps as numbered research problems. Each must be two to three sentences maximum — precise, citable, and falsifiable. Gap 1 — The Permissiveness Problem: GCR and DoG construct constraint oracles as f(G, E_q), independent of question semantics and partial generation. For a 3-hop CWQ question, this produces a valid token set containing up to 24,000 structurally correct but semantically irrelevant paths (from Eq 2.4), forcing the LLM to use hallucination-prone parametric knowledge for discrimination — precisely the mechanism constrained decoding was designed to suppress (Luo et al., 2024; Li et al., 2024). Gap 2 — The Static-Dynamic Efficiency Tradeoff: GCR's precomputed static trie is computationally efficient but semantically blind. DoG's step-wise topological expansion introduces dynamism at the cost of per-step trie reconstruction and without semantic filtering (Li et al., 2024; Willard and Louf, 2023). No existing framework achieves semantic responsiveness while preserving the efficiency of a precomputed oracle. Gap 3 — The Absence of a Permissiveness Metric: no existing work quantifies constraint permissiveness independently of downstream answer accuracy, making it impossible to measure progress on reducing irrelevant paths in the valid set directly or to compare frameworks on this property (Luo et al., 2024; Li et al., 2024).*

🖼️ **Figure**	*Fig 2.14 — Consolidated gap matrix. Rows: GCR, DoG, DCA-Trie v1 (proposed), DCA-Trie v2 (proposed). Columns: Structural Faithfulness (100%), Semantic Relevance Conditioning (Y/N), Dynamic Step-Wise Update (Y/N), Precomputed Oracle Efficiency (Y/N), Permissiveness Quantified via SIR (Y/N). GCR: ✓ ✕ ✕ ✓ ✕. DoG: ✓ ✕ ✓ ✕ ✕. DCA-Trie v1: ✓ ✓ ✕ ✓ ✓. DCA-Trie v2: ✓ ✓ ✓ ◗ ✓. This is the gap matrix that appears as Table 1 in the published paper.*

2.10.2 DCA-Trie as the Unified Response: Gap-to-Objective Mapping

📝 **Writing Note**	*Map each gap to a DCA-Trie component and project objective in structured prose. Gap 1 → DCA-Trie's semantic relevance scorer applied as a hard admission criterion, upgrading the oracle to f(G, E_q, q, y_{<t}) → Objectives ii, iii, iv. Gap 2 → the two-variant design: v1 applies semantic filtering at trie construction time preserving precomputation efficiency; v2 applies step-wise dynamic expansion with semantic filtering for maximum contextual responsiveness → Objectives iii, iv. Gap 3 → the Semantic Irrelevance Ratio (SIR), a new metric introduced in Chapter 3 that measures the proportion of the valid set consisting of semantically irrelevant paths → Objective i. Close with the contribution statement: 'DCA-Trie is the first constrained decoding framework to simultaneously address all three gaps — providing structural faithfulness, semantic relevance conditioning, and a dedicated*

permissiveness metric. These constitute the three novel contributions of this dissertation.'

Figure

Fig 2.15 — DCA-Trie contribution mapping. Four columns: Research Gap | DCA-Trie Component | Project Objective | Evaluated In Chapter. Three rows: Gap 1 (permissiveness) | Semantic scorer + threshold τ | Obj ii, iii, iv | Ch 3–4. Gap 2 (static-dynamic tradeoff) | v1 and v2 variants | Obj iii, iv | Ch 3–4. Gap 3 (no permissiveness metric) | SIR metric | Obj i | Ch 3–4. Navy shading on DCA-Trie Component column.

END OF CHAPTER 2 — LITERATURE REVIEW

Total sections: 10 | Figures required: 15 | Equations required: 11 | Algorithms required: 1

Related Work: 3 subsections (journal-standard) — reduced from 4. Gap matrix: Fig 2.14. Contribution map: Fig 2.15.